
FROM MUJoCo PLAYGROUND TO ROBOTIS OP3: SIM-TO-REAL GAPS IN JOYSTICK LOCOMOTION

EXTENDED ABSTRACT DRAFT

Author One
Affiliation One
email@example.com

Author Two
Affiliation One
email@example.com

Author Three
Affiliation One
email@example.com
Author Four
Affiliation One
email@example.com

May 5, 2026

ABSTRACT

This extended abstract studies whether the MuJoCo Playground zero-shot sim-to-real recipe for humanoid joystick locomotion extends to the smaller ROBOTIS OP3 platform. MuJoCo Playground treats OP3 as a first-class humanoid environment alongside Berkeley Humanoid, Unitree H1/G1, and Booster T1, using the same joystick command abstraction of forward velocity, lateral velocity, and yaw rate. However, the paper and RSS 2025 demonstration evidence report real hardware deployment on larger research platforms, not OP3. Our contribution is therefore not a new reinforcement learning algorithm, but an engineering audit of the path from a canonical Playground OP3 policy to an OP3-facing deployment stack: MJX/Brax PPO training, actuator-order alignment with ROBOTIS motor IDs, ONNX export, ROS 2 policy execution, MuJoCo to Webots sim-to-sim testing, and measurement-driven sim-to-real diagnosis. The central claim is that small mismatches in observation conventions, command filtering, actuator stiffness, and simulator contact assumptions can dominate whether a policy that walks in simulation remains meaningful when moved into a second simulator or onto hardware.

Keywords: ROBOTIS OP3; MuJoCo Playground; Webots; ROS 2; humanoid locomotion; sim-to-sim; sim-to-real; reproducibility.

1 Motivation



High-throughput reinforcement learning can produce visually convincing humanoid locomotion in simulation, but transferring such policies to a physical robot depends on whether the training simulator, exported policy, middleware, and robot control stack describe the same closed-loop system. MuJoCo Playground is a useful test case because it is both a canonical implementation and a strong sim-to-real claim; the authors describe zero-shot transfer across six robot platforms in under eight weeks, with deployment through ONNX Runtime at 50 Hz, ros2-control, C++ real-time control, and a model-based estimator (Zakka et al., 2025; MuJoCo Playground Contributors, 2025a,b).

The raison d'être of MuJoCo Playground is to reduce the friction between simulator construction, policy training, and real-robot deployment. The RSS paper presents Playground as a fully open-source MJX-based framework that lets users install a common robot-learning stack, train policies in minutes on a single GPU, and iterate quickly between simulated environments and hardware. Its design combines GPU-accelerated physics, on-device batch rendering, ready-made locomotion and manipulation environments, reproducible notebooks, hyperparameters, training curves, and deployment examples so that the remaining research effort can focus on robot behaviour and sim-to-real fidelity rather than rebuilding the infrastructure for each platform (Zakka et al., 2025).

Related frameworks make the same infrastructure question timely: Isaac Lab and ROBOTIS Lab provide Isaac-Sim-based robot-learning stacks, while Humanoid-Gym and legged-gym contextualise GPU-parallel locomotion training and sim-to-real transfer (Mittal et al., 2025; ROBOTIS, 2026; Gu et al., 2024; Rudin et al., 2022).

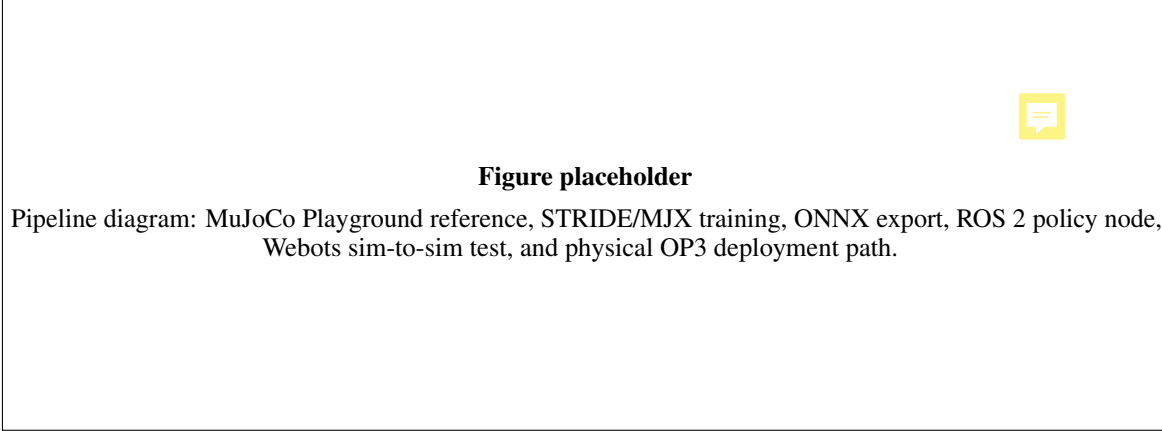


Figure 1: Planned system overview figure.

The ROBOTIS OP3 is present in the Playground humanoid task family, but it is not reported as one of the real-hardware zero-shot platforms in the paper or RSS 2025 demo. This makes OP3 a productive SPECS case study: if the shared joystick abstraction transfers, OP3 provides evidence that the Playground recipe generalises to hobbyist-tier humanoid hardware; if it fails, the failure modes are still informative because they expose deployment assumptions in a widely used canonical library. Our goal is deliberately conservative: reproduce the reference walking behaviour, adapt it to a ROBOTIS-ordered STRIDE pipeline, and identify fidelity gaps before making strong hardware claims.

2 Pipeline

The current pipeline is:

MuJoCo MJCF $\rightarrow$ MJX $\rightarrow$ Brax PPO $\rightarrow$ ONNX $\rightarrow$ ROS 2 $\rightarrow$ Webots / OP3.

The robot is first defined in MuJoCo using an MJCF model, including kinematics, inertial parameters, contact geometry, and position-controlled actuators, following the MuJoCo Playground implementation of the OP3 joystick locomotion task and its MuJoCo Menagerie model source (Zakka et al., 2022, 2025; Google DeepMind, 2025). This model is executed in MJX for accelerated physics simulation, enabling large-scale reinforcement learning with PPO implemented in Brax. The policy runs at 50 Hz with a 147-dimensional observation formed from three stacked 49-dimensional frames. Actions are 20-dimensional joint-position offsets around the default pose. To ensure that the same policy can be deployed across simulators and hardware without modifying the underlying robot description, we introduce a bidirectional mapping layer, termed *Flip-Flop*. The *Flip* component transforms observations from the target system (Webots or physical OP3) into the convention expected by the policy. The *Flop* component performs the inverse transformation on the policy outputs, mapping joint-position commands into the correct target-system actuator ordering. The trained policy is exported to ONNX format to enable deployment outside the JAX runtime. A ROS 2 node loads the ONNX model and performs real-time inference.

3 Sim-to-Sim Transfer

Before deploying on hardware, we introduce a sim-to-sim validation stage in which the trained policy is executed in both MuJoCo (training environment) and Webots (deployment simulator). The purpose is not to claim that Webots is more accurate than MuJoCo, but to test whether the exported ONNX policy and ROS 2 policy node can drive an OP3 model outside the original training simulator. The policy is first verified in MuJoCo to reproduce the reference joystick locomotion behaviour. The same exported ONNX is then deployed in Webots through the ROS 2 policy node. This setup isolates differences arising from simulator dynamics. In particular, differences observed between MuJoCo and Webots can be attributed to actuator modelling (e.g., stiffness and damping), contact dynamics, and timestep integration, rather than joint ordering or coordinate conventions. We compare the two simulators under identical joystick commands using joint position tracking (target versus actual), tracking error over time, torque over time, and step response.

These measurements allow us to identify systematic differences in actuator response. For example, persistent tracking lag after aligning joint stiffness (K_p), damping (K_v), and torque limits in Webots indicates lower effective proportional

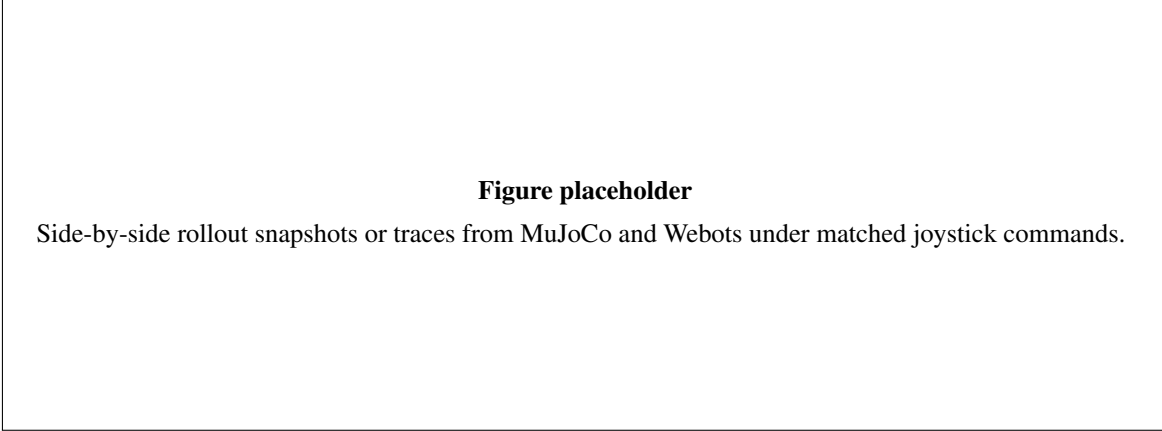


Figure 2: Planned sim-to-sim comparison figure.

gain or torque limits compared to MuJoCo, so differences reflect mismatches in stiffness, damping, and torque-limit values. By resolving these mismatches at the sim-to-sim stage, we reduce the number of noise factors before hardware deployment. Disagreement between MuJoCo and Webots is useful when it localises a deployment assumption that was hidden in the training loop.

The Webots controller is a C++ ROS 2 node running a 20 ms timer callback: read joint states and IMU, assemble the 147-dimensional observation, run the ONNX forward pass, write 20-dimensional joint position targets. A gating counter enforces the 50 Hz policy rate independently of the ROS 2 spin rate. Command velocity is received on the standard `/cmd_vel` topic via `teleop_twist_keyboard`, making the node API-compatible with the planned hardware deployment.

Athul. The paragraph below lists bugs found during integration

Several latent bugs were exposed during integration that would otherwise have appeared only on hardware. (i) *IMU axis convention*: the Webots gyroscope axis mapping was determined empirically by perturbing the robot along each axis and inspecting raw sensor output; assuming documentation accuracy would have left a silent sign error. (ii) *Gravity vector transpose*: `getUpVector()` returns the third *row* of the rotation matrix; using the third column produced a transposed gravity estimate and immediate falls. (iii) *Velocity-limit persistence*: a `setVelocity()` call made during initialisation persists across subsequent `setPosition()` calls, causing unrestricted-speed position commands until the limit is explicitly reset. (iv) *Timestep-dependent gain scaling*: training used $K_d = 1.084$ at a 4 ms physics step; Webots runs at 8 ms, requiring $K_d = 0.2$ for comparable joint damping.

4 Preliminary Findings

Initial reproduction work shows that matching the reference implementation matters more than changing the learning algorithm. Stable walking required Playground-compatible reward scaling, body-frame velocity tracking, observation normalisation, three-frame history, command sampling, and PD gain setup. A custom PPO implementation produced stable standing but poor walking until these details were brought into alignment.

Deployment testing has also exposed control-path issues. The ROBOTIS `direct_control_module` can low-pass filter position commands through minimum-jerk smoothing. For a policy emitting actions every 20 ms, these filtering parameters determine whether the robot receives near-passthrough policy commands or smoothed trajectories with substantially different timing.

5 Sim-to-Real Fidelity Questions

The main simulator mismatch identified so far is actuator stiffness. The default MuJoCo Playground OP3 stiffness is much softer than measurements from the physical robot in a static walk-ready pose. Action-page replay also revealed that trajectory shape matters: linear interpolation in simulation did not match the trapezoidal velocity profile used by

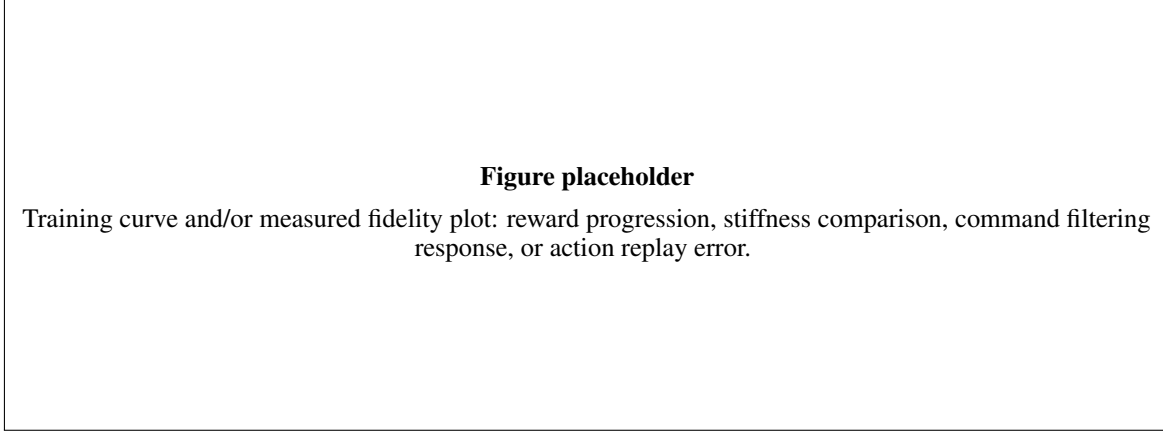


Figure 3: Planned preliminary-results figure.

the ROBOTIS action module. Foot-ground interaction is another open question, because the physical OP3 feet include protruding screws that can behave like high-friction contact points on outdoor surfaces.

6 Contribution

The contribution of this extended abstract is an engineering audit rather than a new locomotion method. We document a reproducible path from a public MuJoCo Playground OP3 task to a ROS 2 deployment stack, add a Webots sim-to-sim stage to test deployment assumptions, and identify the main fidelity gaps that must be resolved before robust physical walking claims are credible. The scientific focus is whether a first-class but not yet reported OP3 Playground environment can support the same zero-shot or near-zero-shot deployment narrative demonstrated on larger platforms.

Athul. Added info in contributions

Concretely: actuator-order alignment between MuJoCo indices and ROBOTIS motor IDs 1–20; ONNX export and C++ ROS 2 integration of a JAX/Brax policy; a perturbation-based IMU remapping procedure transferable to other simulators and hardware; and a fidelity checklist covering actuator stiffness, contact geometry, command filtering, and timestep mismatch.

Acknowledgements

Placeholder for funding, lab, robot access, and acknowledgements.

References

- Google DeepMind. MuJoCo Playground: Gpu-accelerated robot learning and sim-to-real transfer. https://github.com/google-deepmind/mujoco_playground, 2025. Accessed 2026-05-04.
- Xinyang Gu, Yen-Jen Wang, and Jianyu Chen. Humanoid-Gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer. *arXiv preprint arXiv:2404.05695*, 2024. URL <https://arxiv.org/abs/2404.05695>.
- Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrugg, Nikita Rudin, Lukasz Wawrzyniak, Milad Rakhsha, Alain Denzler, Eric Heiden, Ales Borovicka, Ossama Ahmed, Ireaiyo Akinola, Abrar Anwar, Mark T. Carlson, Ji Yuan Feng, et al. Isaac Lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. URL <https://arxiv.org/abs/2511.04831>.
- MuJoCo Playground Contributors. MuJoCo Playground project site. <https://playground.mujoco.org/>, 2025a. Accessed 2026-05-04.
- MuJoCo Playground Contributors. MuJoCo Playground at RSS 2025. <https://playground.mujoco.org/demo/>, 2025b. Accessed 2026-05-04.

- ROBOTIS. ROBOTIS Lab: Reinforcement learning and imitation learning for ROBOTIS robots. https://github.com/ROBOTIS-GIT/robotis_lab, 2026. Accessed 2026-05-05.
- Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the Conference on Robot Learning*, pages 91–100. PMLR, 2022. URL <https://arxiv.org/abs/2109.11978>.
- Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high quality simulation models for MuJoCo. https://github.com/google-deepmind/mujoco_menagerie, 2022. Accessed 2026-05-01.
- Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder Alexander Kahrs, Carmelo Sferrazza, Yuval Tassa, and Pieter Abbeel. Demonstrating MuJoCo Playground. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.020. URL <https://www.roboticsproceedings.org/rss21/p020.html>.